src/config.py

Every tuned constant in one place, each with a reason

This module is the project's control panel. Every judgement parameter — a path, a model name, a chunk size, a loop cap — is a named constant here with a prose justification, so no answer-changing number is ever a magic value buried in a function.

Walkthrough of all 182 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Central configuration for the agentic RAG research assistant.
 2
 3  Every judgement parameter in this project is a named constant here, each with a
 4  prose "why". The rule (borrowed from the rest of this portfolio): if a number
 5  changes the answer, it must be visible and defensible — never a magic number
 6  buried in a function. `run_pipeline.py`, `run_eval.py`, the notebooks, and the
 7  tests all read their settings from this one module, so there is a single source
 8  of truth for how the system is tuned.
 9  """
```

Lines 1 to 9 are the module docstring, stating the governing rule: if a number changes the answer it must be visible and defensible here, and every entry point (pipeline, eval, notebooks, tests) reads its settings from this one module.

```
10
11  from __future__ import annotations
12
13  import os
14  from pathlib import Path
```

Lines 10 to 14 import os (to read the API-key environment variable) and Path (for building filesystem paths), after enabling postponed annotations on line 11.

```
15
16  # ---------------------------------------------------------------------------- #
17  # Determinism
18  # ---------------------------------------------------------------------------- #
19  # One seed for the whole project. Embedding a query, sampling papers, shuffling
20  # an evaluation set — all of it is reproducible so every published number can be
21  # regenerated exactly. Matches the SEED = 42 convention used across the portfolio.
22  SEED = 42
```

Lines 15 to 22 define SEED = 42. The comment explains it makes every stochastic step — embedding, sampling, shuffling — reproducible, matching the portfolio-wide seed convention.

```
23
24  # ---------------------------------------------------------------------------- #
25  # Paths — everything is resolved relative to the project root, so the code runs
26  # the same whether invoked from a notebook, a test, or the command line.
27  # ---------------------------------------------------------------------------- #
28  PROJECT_ROOT = Path(__file__).resolve().parent.parent
29  DATA_DIR = PROJECT_ROOT / "data"
30  RAW_DIR = DATA_DIR / "raw"
31  PROCESSED_DIR = DATA_DIR / "processed"
32  INDEX_DIR = DATA_DIR / "index"
33  OUTPUTS_DIR = PROJECT_ROOT / "outputs"
34  FIGURES_DIR = OUTPUTS_DIR / "figures"
35  RESULTS_DIR = OUTPUTS_DIR / "results"
36  EVAL_DIR = PROJECT_ROOT / "eval"
```

Lines 23 to 36 define the directory paths. Line 28 anchors PROJECT_ROOT to the folder two levels above this file, so paths resolve identically from a notebook, a test, or the command line. Lines 29 to 36 derive the data, raw, processed, index, outputs, figures, results, and eval directories from that root using the path-join operator.

```
37
38  # Named artifact files. Referenced by the pipeline and every consumer so a
39  # rename happens in exactly one place.
40  PROVENANCE_FILE = RAW_DIR / "provenance.json"
41  PAPERS_FILE = RAW_DIR / "papers.jsonl"          # one arXiv paper per line
42  CHUNKS_FILE = PROCESSED_DIR / "chunks.parquet"  # one chunk per row
43  FAISS_INDEX_FILE = INDEX_DIR / "faiss.index"    # the serialized vector index
44  CHUNK_META_FILE = INDEX_DIR / "chunk_meta.parquet"  # row-aligned with the index
```

Lines 37 to 44 name the individual artifact files — provenance JSON, the papers JSONL, the chunks parquet, the FAISS index, and the row-aligned chunk-metadata parquet — so a rename happens in exactly one place. The inline comments document each file's shape.

```
45
46  # ---------------------------------------------------------------------------- #
47  # Corpus acquisition (arXiv)
48  # ---------------------------------------------------------------------------- #
49  # We retrieve recent NLP / ML papers. cs.CL is computational linguistics and
50  # cs.LG is machine learning — the two categories most likely to contain the
51  # retrieval, embedding, and agent literature our evaluation questions ask about,
52  # so the corpus actually *can* answer them.
53  ARXIV_CATEGORIES = ("cs.CL", "cs.LG")
```

Lines 45 to 53 set ARXIV_CATEGORIES to cs.CL and cs.LG. The comment explains these are computational linguistics and machine learning — the categories most likely to contain the retrieval, embedding, and agent literature the evaluation asks about, so the corpus can actually answer the questions.

```
54
55  # How many papers to pull. Bounded deliberately: the whole pipeline (download →
56  # chunk → embed → index) must finish in a few minutes on a CPU laptop so the
57  # capstone notebook can run top-to-bottom. A few thousand abstracts is plenty to
58  # make retrieval quality visible without a GPU or an overnight job.
59  MAX_PAPERS = 2000
```

Lines 54 to 59 set MAX_PAPERS = 2000. The comment justifies the bound: the full pipeline must finish in minutes on a CPU so the capstone can run top-to-bottom, and a few thousand abstracts is enough to make retrieval quality visible.

```
60
61  # arXiv asks clients to page politely and sleep between requests. These match
62  # their published guidance; raising the page size or dropping the delay risks a
63  # temporary IP block, which would make the pipeline *less* reproducible.
64  ARXIV_PAGE_SIZE = 100
65  ARXIV_REQUEST_DELAY_SECONDS = 3.0
```

Lines 60 to 65 set ARXIV_PAGE_SIZE = 100 and ARXIV_REQUEST_DELAY_SECONDS = 3.0. The comment notes these match arXiv's published rate guidance; being greedier risks a temporary IP block that would hurt reproducibility.

```
66
67  # Seminal papers, fetched by id and merged into the corpus regardless of the
68  # recent slice. Two reasons this matters:
69  #   1. The evaluation set's ground-truth answers point at these papers, so they
70  #      must be present for the metrics to be meaningful. Pinning them by id makes
71  #      the evaluation deterministic — it does not depend on which recent papers
72  #      happened to be returned that day.
73  #   2. They are the papers the retrieval/embedding/agent literature is *about*,
74  #      so a corpus that contains them can actually answer questions about the
75  #      field's foundations.
76  SEED_PAPER_IDS = (
77      "1706.03762",  # Attention Is All You Need (the Transformer)
78      "2005.11401",  # Retrieval-Augmented Generation (the original RAG paper)
79      "1810.04805",  # BERT
80      "2005.14165",  # GPT-3 (Language Models are Few-Shot Learners)
81      "1908.10084",  # Sentence-BERT (the embedding model family we use)
82      "2004.04906",  # Dense Passage Retrieval (DPR)
83      "2210.03629",  # ReAct (reason + act — the agentic-loop idea)
84      "2201.11903",  # Chain-of-Thought prompting
85      "1301.3781",   # word2vec (Efficient Estimation of Word Representations)
86      "1409.0473",   # Bahdanau attention (align-and-translate)
87      "2203.02155",  # InstructGPT (training LMs to follow instructions with RLHF)
88      "2302.13971",  # LLaMA
89  )
```

Lines 66 to 89 define SEED_PAPER_IDS, a tuple of twelve seminal papers fetched by id regardless of the recent slice. The comment (lines 67 to 75) gives two reasons: the evaluation's ground-truth answers point at these papers so they must always be present (making the metrics deterministic), and they are the foundational works the field is about. Each id (lines 77 to 88) carries a comment naming the paper — the Transformer, RAG, BERT, GPT-3, Sentence-BERT, DPR, ReAct, chain-of-thought, word2vec, Bahdanau attention, InstructGPT, and LLaMA.

```
 90
 91  # ---------------------------------------------------------------------------- #
 92  # Chunking
 93  # ---------------------------------------------------------------------------- #
 94  # Retrieval works on *chunks*, not whole documents. Too-large chunks dilute the
 95  # embedding (one vector trying to represent several ideas); too-small chunks lose
 96  # the context a sentence needs to be meaningful. ~1000 characters is a few
 97  # sentences — one coherent idea — which is a well-trodden default for abstract-
 98  # and-paragraph-length scientific text.
 99  CHUNK_SIZE_CHARS = 1000
```

Lines 90 to 99 set CHUNK_SIZE_CHARS = 1000. The comment explains the trade-off: too-large chunks dilute the embedding across several ideas, too-small ones lose context, and about 1000 characters is one coherent idea for scientific text.

```
100
101  # Overlap carries a little context across the boundary so a fact split across two
102  # chunks is still retrievable from at least one of them. 150 chars (~15%) is the
103  # usual trade-off: enough to avoid orphaning a sentence, small enough not to
104  # balloon the index with near-duplicate text.
105  CHUNK_OVERLAP_CHARS = 150
```

Lines 100 to 105 set CHUNK_OVERLAP_CHARS = 150. The comment explains overlap (~15%) carries context across a boundary so a fact split between two chunks survives in at least one, without ballooning the index with near-duplicate text.

```
106
107  # ---------------------------------------------------------------------------- #
108  # Embeddings
109  # ---------------------------------------------------------------------------- #
110  # all-MiniLM-L6-v2 is the workhorse sentence-transformer: 384-dimensional, fast
111  # on CPU, and strong on short-text semantic similarity — exactly this task. It is
112  # small enough to download and run in the capstone without a GPU, which keeps the
113  # whole retrieval path reproducible on any machine.
114  EMBED_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
115  EMBED_DIM = 384
116  EMBED_BATCH_SIZE = 64
```

Lines 106 to 116 configure embeddings: EMBED_MODEL is all-MiniLM-L6-v2 (line 114), EMBED_DIM is 384 (line 115), and EMBED_BATCH_SIZE is 64 (line 116). The comment justifies the model as a fast, CPU-friendly, short-text sentence-transformer that keeps the retrieval path reproducible without a GPU.

```
117
118  # ---------------------------------------------------------------------------- #
119  # Retrieval
120  # ---------------------------------------------------------------------------- #
121  # How many chunks a single retrieval call returns. 5 is a common default: enough
122  # context for the LLM to synthesize an answer, few enough that an irrelevant
123  # chunk can't crowd out the relevant ones ("lost in the middle"). The agent can
124  # retrieve multiple *times*; this is the width of one retrieval.
125  TOP_K = 5
```

Lines 117 to 125 set TOP_K = 5. The comment explains 5 is enough context for the LLM to synthesize an answer but few enough that an irrelevant chunk cannot crowd out the relevant ones (the lost-in-the-middle

problem); the agent can retrieve multiple times, so this is the width of a single retrieval.

```
126
127  # ---------------------------------------------------------------------------- #
128  # Generation + agent (Claude via the Anthropic API)
129  # ---------------------------------------------------------------------------- #
130  # The generation model. Sonnet 5 is the cost/quality sweet spot for grounded
131  # question-answering over retrieved context — near-Opus quality at Sonnet price.
132  # Swap to "claude-opus-5" for maximum quality or "claude-haiku-4-5" to cut cost;
133  # every call reads this constant, so it changes in one place.
134  GEN_MODEL = "claude-sonnet-5"
```

Lines 126 to 134 set GEN_MODEL to claude-sonnet-5. The comment frames it as the cost/quality sweet spot for grounded question-answering and notes it can be swapped to Opus for maximum quality or Haiku to cut cost, in this one place.

```
135
136  # The judge model for the answer-quality evaluation (LLM-as-judge). Kept the same
137  # as the generation model by default so the grader is at least as capable as the
138  # thing it grades.
139  JUDGE_MODEL = "claude-sonnet-5"
```

Lines 135 to 139 set JUDGE_MODEL, the LLM-as-judge model for answer evaluation, to the same model as generation so the grader is at least as capable as what it grades.

```
140
141  # Hard ceiling on answer length. Generous enough for a cited multi-paragraph
142  # answer, bounded so a runaway generation can't rack up cost.
143  MAX_ANSWER_TOKENS = 1024
```

Lines 140 to 143 set MAX_ANSWER_TOKENS = 1024 — generous enough for a cited multi-paragraph answer but bounded so a runaway generation cannot rack up cost.

```
144
145  # The agentic loop's stopping condition. The agent may call the retrieval tool,
146  # read what it got, decide it needs more, and search again — but not forever.
147  # After this many tool-use rounds it must answer with what it has. 4 is enough
148  # for a two- or three-hop question (decompose → search → search again → answer)
149  # without letting a confused agent loop indefinitely and burn tokens.
150  MAX_AGENT_STEPS = 4
```

Lines 144 to 150 set MAX_AGENT_STEPS = 4, the agent loop's stopping condition. The comment explains 4 rounds is enough for a two- or three-hop question (decompose, search, search again, answer) without letting a confused agent loop forever.

```
151
152  # Where the on-disk LLM response cache lives. Every Claude call is cached by a
153  # hash of its inputs so (a) the notebook and tests reproduce published numbers
154  # WITHOUT an API key, and (b) re-running the pipeline doesn't pay for identical
155  # calls twice. This is the same "reference-only, load the saved result"
156  # convention the portfolio's Kaggle-GPU notebooks use.
157  LLM_CACHE_FILE = RESULTS_DIR / "llm_cache.json"
```

Lines 151 to 157 set LLM_CACHE_FILE. The comment explains every Claude call is cached by an input hash so the notebook and tests reproduce published numbers without a key, and re-runs never pay twice for identical calls.

```
158
159  # The environment variable holding the Anthropic API key. NEVER hard-code a key;
160  # NEVER commit one. If it is unset, live LLM calls are skipped and the cache is
161  # used instead — so the project degrades gracefully to "reproduce from cache".
162  ANTHROPIC_API_KEY_ENV = "ANTHROPIC_API_KEY"
```

Lines 158 to 162 set ANTHROPIC_API_KEY_ENV to the name of the environment variable holding the key. The comment stresses never hard-coding or committing a key, and that an unset key makes the project fall back to the cache.

```
163
164
165  def api_key_available() -> bool:
166      """True when an Anthropic API key is present in the environment.
167
168      Used everywhere a live call is optional: if this is False we fall back to the
169      committed response cache instead of erroring, which is what lets the whole
170      project reproduce on a machine with no key.
171      """
172      return bool(os.environ.get(ANTHROPIC_API_KEY_ENV))
```

Lines 163 to 172 define api_key_available(), returning True when the key environment variable is set (line 172 wraps os.environ.get in bool, so an empty value counts as absent). The docstring explains this gate is what lets live calls be optional and the project reproduce on a machine with no key.

```
173
174
175  def ensure_dirs() -> None:
176      """Create every output/data directory this project writes to.
177
178      Called at the top of the pipeline and eval entry points so a fresh checkout
179      (where data/ and outputs/ are git-ignored and therefore absent) just works.
180      """
181      for d in (RAW_DIR, PROCESSED_DIR, INDEX_DIR, FIGURES_DIR, RESULTS_DIR):
182          d.mkdir(parents=True, exist_ok=True)
```

Lines 173 to 182 define ensure_dirs(), which creates every directory the project writes to. The loop (lines 181 to 182) calls mkdir with parents=True and exist_ok=True on the raw, processed, index, figures, and results directories, so a fresh checkout (where data/ and outputs/ are git-ignored and absent) just works.